Chapter 2

Instructions: Language of the Computer

Instruction Set

- The repertoire of instructions of a computer
- Different computers have different instruction sets
  - But with many aspects in common
- Early computers had very simple instruction sets
  - Simplified implementation
- Many modern computers also have simple instruction sets

The MIPS Instruction Set

- Used as the example throughout the book
- Stanford MIPS commercialized by MIPS Technologies (www.mips.com)
- Typical of many modern ISAs
  - See MIPS Reference Data tear-out card, and Appendixes B and E
- Similar ISAs have a large share of embedded core market
  - Applications in consumer electronics, network/storage equipment, cameras, printers, …

Arithmetic Operations

- Add and subtract, three operands
  - Two sources and one destination

Q1. Given the MIPS instruction:
add a, b, c
Match each component with its correct meaning.

Component	Meaning
a	A. Operation to be performed
b	B. Second source operand
c	C. Destination operand
add	D. First source operand

A1. C, D, B, A
add a, b, c means a = b + c

Arithmetic Example

- C code:

```
f = (g + h) - (i + j);
```

Q2. Given f = (g - h) + (i - j), complete the MIPS instructions.
sub ____, g, h
sub ____, i, j
add f, ____, ____

A2. t0, t1, t0, t1

Register Operands

- Arithmetic instructions use register operands
- MIPS has a 32 × 32-bit register file
  - Use for frequently accessed data
  - Numbered 0 to 31
  - 32-bit data called a “word”
- Assembler names
  - $t0, $t1, …, $t9 for temporary values
  - $s0, $s1, …, $s7 for saved variables
- *Design Principle 2:* Smaller is faster
  - c.f. main memory: millions of locations

Register Operand Example

- C code:

```
f = (g + h) - (i + j);
```

Q3. Given the initial register values:

Register	Initial Value
$s1	10
$s2	6
$s3	8
$s4	3

Execute the following MIPS instructions

```
add $t0, $s1, $s2
add $t1, $s3, $s4
sub $s0, $t0, $t1
```

What are the final values of $t0, $t1, and $s0?

A3.

$t0 = 10 + 6 = 16

$t1 = 8 + 3 = 11

$s0 = 16 - 11 = 5

Memory Operands

- Main memory used for composite data
  - Arrays, structures, dynamic data
- To apply arithmetic operations
  - Load values from memory into registers
  - Store result from register to memory
- Memory is byte addressed
  - Each address identifies an 8-bit byte
- Words are aligned in memory
  - Address must be a multiple of 4
- MIPS is Big Endian
  - Most-significant byte at least address of a word
  - *c.f.* Little Endian: least-significant byte at least address

Memory Operand Example 1

- C code:

Q4. Assume $s3 contains the base address of integer array A. Each element is one word (4 bytes). Load A[6] into $t0.

(a) What is the byte offset of A[6]?

(b) What is the effective address if $s3 = 1000?

(c) Write the MIPS instruction.

A4.

Offset=6×4=24

Effective Address=1000+24=1024

lw $t0, 24($s3)

Memory Operand Example 2

- C code:

Q5. Assume:
$s3 contains the base address of integer array A.
h is stored in $s2, and $s2 = 10.
A[4] = 7.
Each array element is one word (4 bytes).
Translate the following statement into MIPS instructions:
A[6] = h + A[4];
Then determine the final value of A[6]

A5.

```
lw  $t0, 16($s3)     # $t0 = A[4] = 7
add $t0, $s2, $t0    # $t0 = 10 + 7 = 17
sw  $t0, 24($s3)     # A[6] = 17
```

Registers vs. Memory

Q6. Consider the two cases for computing: c=a+b

	Case A: Registers	Case B: Memory
Assumption	a, b, and c reside in registers.	a, b, and c reside in memory.
MIPS Instructions	add $s2, $s0, $s1	lw $t0, 0($s0) lw $t1, 4($s0) add $t2, $t0, $t1 sw $t2, 8($s0)

Instructions required in each case:

Memory-access instructions required in each case:

Which case is generally faster, and why?

A6.

	Case A: Registers	Case B: Memory
# Instructions	1	4
# Memory-Access Instructions	0	3 (2 lw + 1 sw)
Generally Faster?	Yes	No

Immediate Operands

- Constant data specified in an instruction

```
addi $s3, $s3, 4
```

Q7. Consider the following MIPS instructions:

```
add  $t0, $s1, $s2
addi $t1, $s1, 10
addi $t2, $t1, -4
```

(a) Which instructions contain an immediate operand?

(b) What is the immediate value in each of these instructions?

(c) Where are these immediate values stored?

A7.

(a) 2nd and 3rd Instructions

(b) The immediate values are 10 in 2nd and –4 in 3rd

(c) Encoded directly in the instructions

The Constant Zero

Q8. Assume: $s1=15, $t0=8 Execute the following instructions in order:

add $t1, $s1, $zero
add $zero, $t0, $s1
add $t2, $zero, $t0

(a) What is the final value of $t1?
(b) What is the final value of $zero?
(c) What is the final value of $t2?

A8.

(a) $t1 = 15, because $zero always provides the value 0.
(b) $zero=0. Writing a result to $zero doesn't change its value.

Section 2-1 Quick Questions

Q1. For “add a, b, c”, match add, a, b, and c with operation, destination, and source operands.

Q2. Complete the MIPS instructions for: f = (g − h) + (i − j).

Q3. Given initial register values, execute the instruction sequence and determine the final values of $t0, $t1, and $s0.

Q4. For word array A, load A[6] into $t0. Determine the byte offset and write the lw instruction.

Section 2-1 Quick Questions

Q5. Translate A[6] = h + A[4] into MIPS using load–operate–store, then determine the final value of A[6].

Q6. For c = a + b, compare the instruction count and memory accesses when values reside in registers versus memory.

Q7. Identify which instructions use immediate operands, state the immediate values, and explain where they are stored.

Q8. Given a short instruction sequence using $zero, determine the final values of $t1, $zero, and $t2.

Exercise 1 | Arithmetic & Register Operands

Assume: f → $s0, g → $s1, h → $s2, i → $s3, j → $s4
Translate the C statement into MIPS instructions:
f = (g − h) + (i + j) − g

(a) Write the MIPS instruction sequence using $t0, $t1, … as needed.
(b) If $s1 = 8, $s2 = 3, $s3 = 6, and $s4 = 4, find the final value of $s0.

Answer

(a)
```
sub $t0, $s1, $s2
add $t1, $s3, $s4
add $t0, $t0, $t1
sub $s0, $t0, $s1
```

(b) $t0 = 8 − 3 = 5, $t1 = 6 + 4 = 10
$t0 = 5 + 10 = 15, $s0 = 15 − 8 = 7

Key Point: Complex expressions are decomposed into simple MIPS arithmetic instructions, with intermediate results stored in temporary registers.

Exercise 2 | Register Value Tracking

Assume: $s0 = 12, $s1 = 5, $s2 = 8.
Execute the following instructions in order:

```
add $t0, $s0, $s1
sub $t1, $t0, $s2
add $t2, $t1, $s1
sub $s3, $t2, $t0
```

(a) Determine the final values of $t0, $t1, $t2, and $s3.
(b) Which registers contain intermediate results?

Answer

(a) $t0 = 12 + 5 = 17
$t1 = 17 − 8 = 9
$t2 = 9 + 5 = 14
$s3 = 14 − 17 = −3
(b) $t0, $t1, and $t2 contain intermediate results.

Key Point: MIPS instructions execute sequentially, and later instructions can use results produced by earlier instructions.

Exercise 3 | Array Addressing and Memory Access

Assume $s3 = 2000 contains the base address of integer array A. Each element is one word (4 bytes).

(a) Calculate the byte offsets of A[5] and A[9].
(b) Calculate their effective memory addresses.
(c) Write MIPS instructions to load A[5] into $t0 and A[9] into $t1.

Answer

(a) A[5]: 5 × 4 = 20 bytes
A[9]: 9 × 4 = 36 bytes
(b) Address of A[5] = 2000 + 20 = 2020
Address of A[9] = 2000 + 36 = 2036
(c) lw $t0, 20($s3)
lw $t1, 36($s3)

Key Point: For a word array, byte offset = array index × 4, and effective address = base address + offset.

Exercise 4 | Load–Operate–Store

Assume $s3 contains the base address of integer array A.
h is stored in $s2, $s2 = 6, and A[3] = 10.
Translate: A[7] = A[3] − h
(a) Calculate the required byte offsets.
(b) Write the MIPS instruction sequence.
(c) Determine the final value of A[7].

Answer

(a) A[3]: 3 × 4 = 12 bytes
A[7]: 7 × 4 = 28 bytes
(b)
```
lw  $t0, 12($s3)
sub $t0, $t0, $s2
sw  $t0, 28($s3)
```
(c) A[7] = 10 − 6 = 4

Key Point: Load memory data into a register, perform arithmetic on registers, then store the result back to memory.

Exercise 5 | Integrated MIPS Analysis

Assume $s3 contains the base address of integer array A, A[2] = 9, and $s1 = 5.

Execute:

```
lw   $t0, 8($s3)
addi $t0, $t0, 4
add  $t1, $t0, $s1
add  $s2, $t1, $zero
sw   $s2, 16($s3)
```

Answer

(a) $t0 = A[2] = 9

(b) $t0 = 9 + 4 = 13; $t1 = 13 + 5 = 18; $s2 = 18

(c) 16 ÷ 4 = 4, so A[4] = 18

(d) Immediate operand: 4. $zero always supplies 0, so add $s2, $t1, $zero copies the value in $t1 to $s2.

Key Point: MIPS combines register operations, memory accesses, immediate constants, and $zero to implement complex computations.